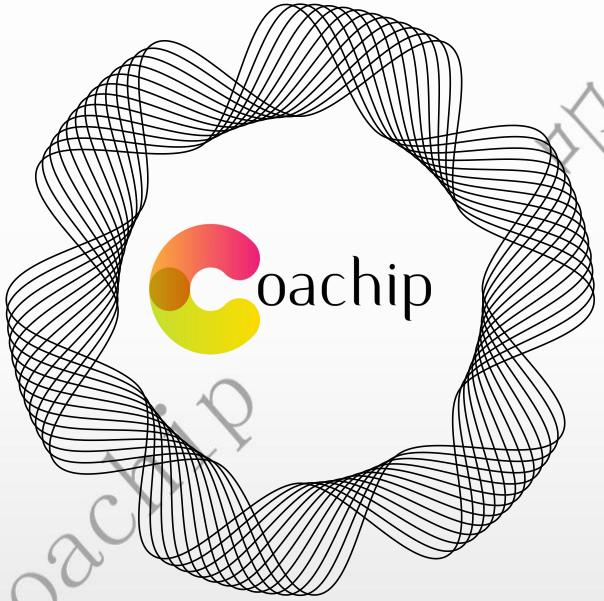


SV面向对象编程

叩特电子Coachip

课程内容及目标

课程内容：

- 类的成员
- 创建对象
- 构造函数

目标：

- 了解类的成员、创建对象、构造函数
- 了解如何使用成员、创建对象



叩特电子Coachip

System Verilog类的成员

- 类是成员变量和成员方法的载体，一个类的功能应该尽可能简单，不应当承担过多的职责，更不应该承担不符合它的职责，这在设计模式中称之为单一职责原则（SRP Single Responsibility Principle）。
- 类作为载体，也具备了天生的闭合属性，即将其属性和方法封装在内部，不会直接将成员变量暴露给外部，通过**protected**和**local**关键词来设置成员变量和方法的外部访问权限。

叩特电子Coachip

System Verilog类的成员

```
class clock
  local bit is_summer = 0;
  local int nclock = 6;
  function int get_clock();
    if(!is_summer) return
this.nclock;
    else return this.nclock+1;
  endfunction
  function bit set_summer(bit s);
    this.is_summer = s;
  endfunction
endclass
```

```
clock ck;
initial begin
  ck = new();
  $display("now time is %0d",
ck.get_clock());
  ck.set_summer(1);
  $display("now time is %0d", ck.nclock);
end
```

** Error (suppressible): (vlog-8688) : Illegal access to local member nclock.

叩特电子Coachip

System Verilog类的成员

- 如果没有指明访问类型，那么成员的默认类型是**public**，子类和外部均可以访问成员。
- 如果指明了访问类型是**protected**，那么只有该类或者子类可以访问成员，而外部无法访问。
- 如果指明了访问类型是**local**，那么只有该类可以访问成员，子类和外部均无法访问。
- 访问类型的设定是为了更好地封装类，尤其是要发布供他人使用的软件包（商业软件），但对于初学者以及应用范围较窄的验证环境，可以使用默认的访问类型，以便于在类的内部或者外部更方便地修改成员变量或者调用成员方法。

叩特电子Coachip

创建对象

- 创建对象？它创建了什么呢？
  - 开辟了新的内存空间，用来存放新的成员变量和方法
- 再次强调Verilog的例化和SV class例化的差别：
  - 两者的共同点在于使用相同的‘模板’来创建内存实例
  - 不同点在于Verilog的例化是静态的，即在编译链接时完成，而SV class的例化是动态的，可以在任意时间点发生，这也使得类的例化方式更加灵活和节省空间。
  - Verilog中没有句柄的概念，即只能通过层次化的索引方式A.B.sigX，而SV class通过句柄可以将对象的指针赋予其它句柄，使得操作更加灵活。
- 在创建对象时，需要注意什么是‘声明’，什么是‘创建’（即例化）

```
Transaction tr; // 声明句柄，此时被初始化为null
tr = new();     // 创建对象，将变量初始化为默认值(二值变量为0,四值变量为x)
```

叩特电子Coachip

定制构造函数(Constructor)

- new()函数,也称构造函数
  - 可以通过自定义new函数将初始化的默认值改成想要的值.
  - New函数没有返回值，因为构造函数总是返回一个指向类对象的句柄，其类型就是类本身.

```
class Transaction;
  logic [31:0] addr,crc,data[8];

  function new;
      addr=3;
      foreach(data[i])
            data[i]=5;
  endfunction
endclass
```

addr和data设为固定值,但crc仍然被初始化为X.

由此可见，可以使用具有默认值的函数来创建更加灵活的构造函数;

叩特电子Coachip

带参数的new函数

```
class Transaction;
  logic [31:0] addr,crc,data[8];

    function new(logic[31:0]a=3,d=5);
        addr=a;
        foreach(data[i])
              data[i]=d;
    endfunction
endclass

initial begin
    Transaction tr;
    tr=new(10);
end

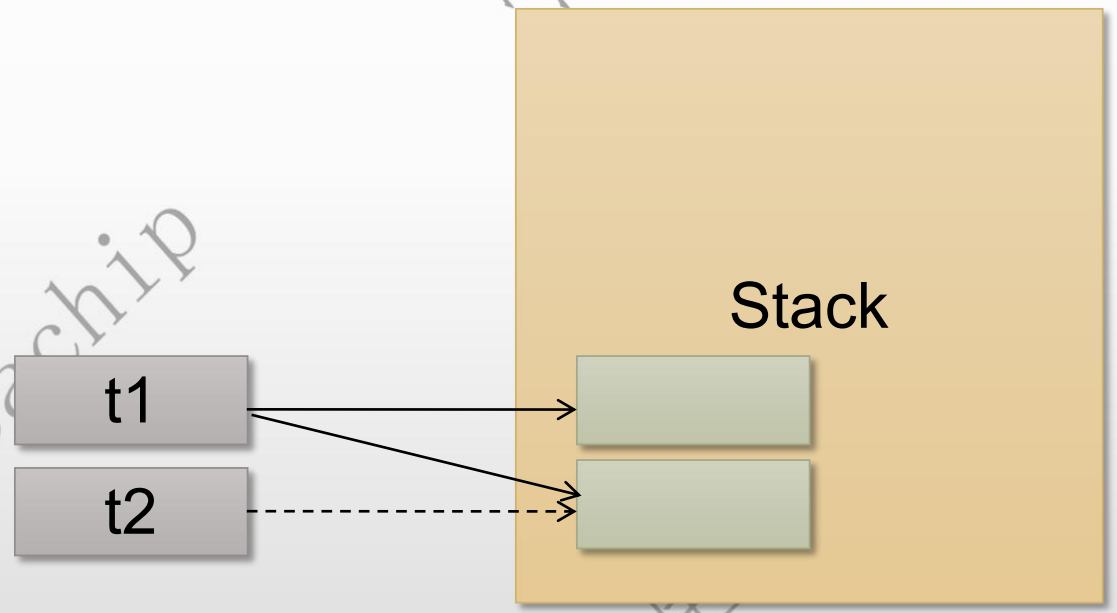
tr = new(.a(10));//a=10,d uses default of 5 tr=new(10);
```

叩特电子Coachip

句柄的传递

句柄可以用来创建多个对象，也可以前后指向不同对象

```
Transaction t1,t2; // 声明句柄
t1 = new(); // 创建对象并将其指针赋予t1
t2 = new(); // 创建对象并将其指针赋予t2
t1 = t2;
// 将t2的值赋予t1,t1和t2指向同一对象，
// t1之前指向的对象被释放
t2 = null;
// 将t2赋值为‘空’即不指向任何对象，
// 此时指针‘悬空’，悬空的指针很‘危险’
```



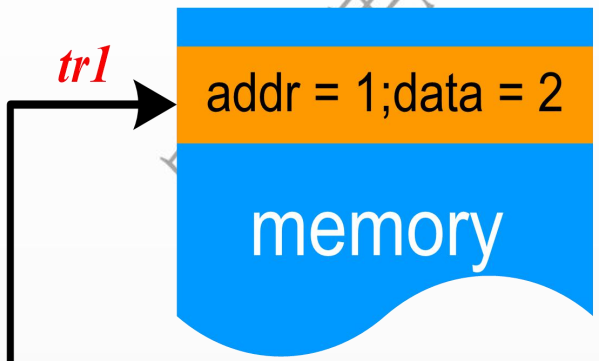
即特电子Coachip

对象的复制

在SystemVerilog中，当一个class声明时会指定一个句柄handle（可以理解为C语言中的指针），该***句柄handle指向一块存储空间，该存储空间后续只能存放该class数据类型的数据。***在没有对其进行任何操作之前，该handle相当于没有指向任何实际的数据对象，只有在使用了new函数之后，才会为其开辟具体的存储对应class类型数据的存储空间。

```
// delacare the handles
transaction tr1;
transaction tr2;

// construct the object
tr1 = new();
tr1.addr = 1;
tr1.data = 2;
```



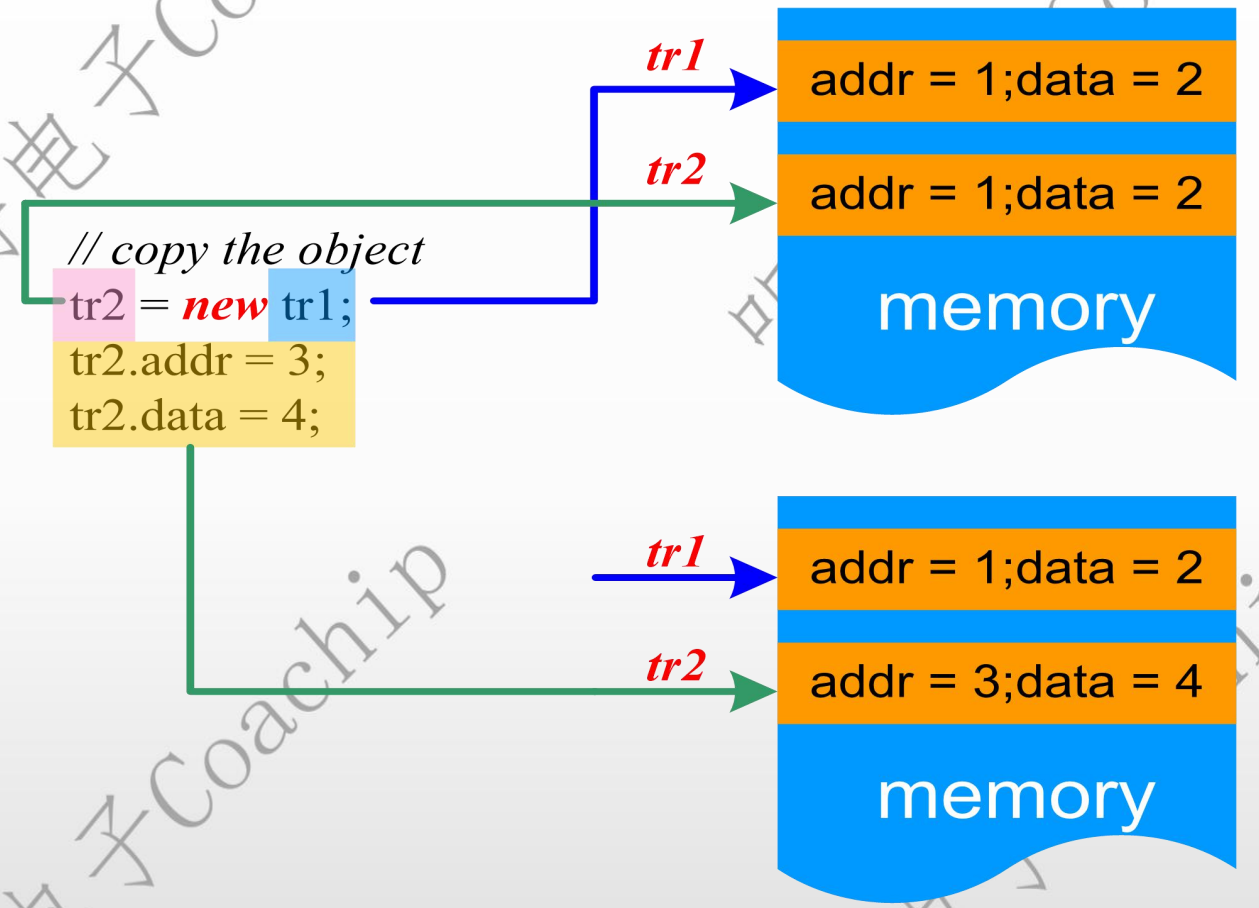
即特电子Coachip

对象的复制-shallow copy

- 常用的浅复制方法有：

  tr2 = new tr1;

- 浅复制复制对象的所有属性；
- 浅复制不会调用new函数；



叩特电子Coachip

对象的复制-shallow copy

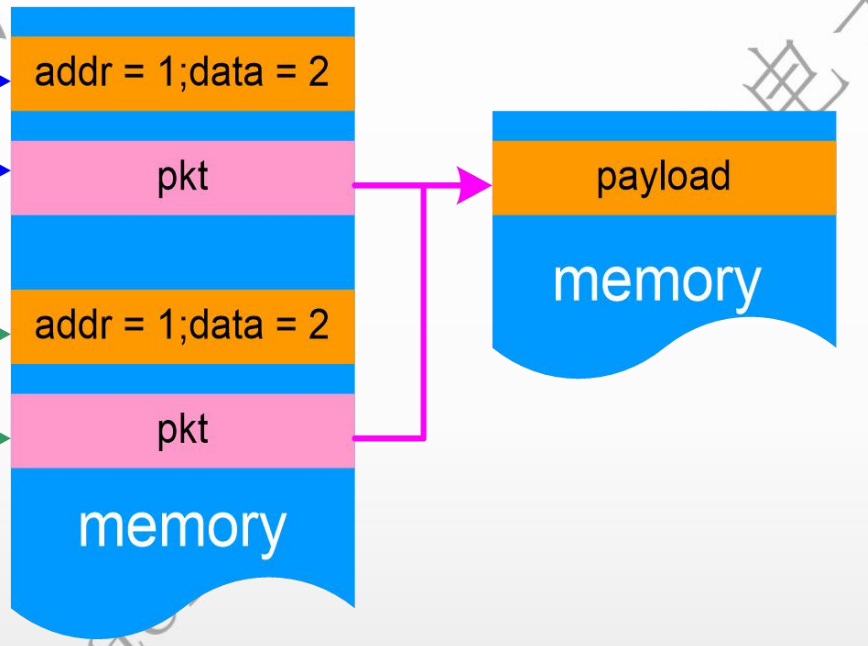
```
class packet;
   bit [7:0] payload;
endclass

class transaction;
bit [7:0] addr;
bit [7:0] data;
packet   pkt;

function new();
   pkt = new();
endfunction
endclass
```

```
transaction tr1;
transaction tr2;

tr1 = new();
tr2 = new tr1;
```



浅复制仅复制对象的所有属性，但不包括属性成员中句柄指向的对象！

对象的复制-deep copy

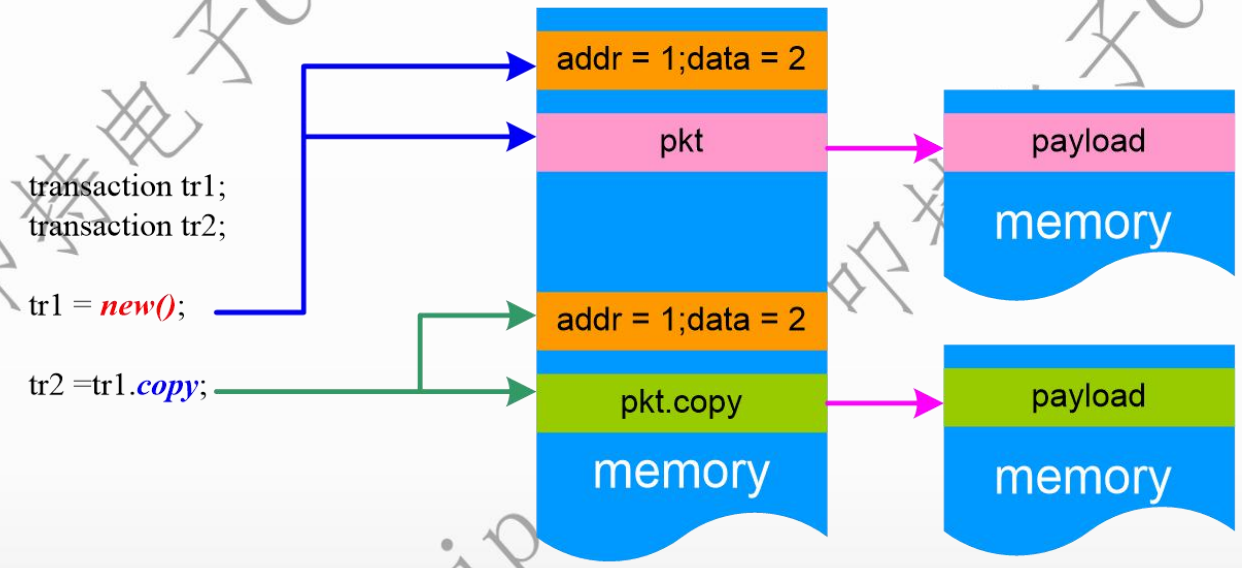
```
class packet;
  bit [7:0] payload;

function packet copy;
   copy = new();
   copy.payload = this.payload;
   return copy;
endfunction
endclass
```

```
class transaction;
bit [7:0] addr;
bit [7:0] data;
packet   pkt;

function new();
   pkt = new();
endfunction

function transaction copy;
   copy = new();
   copy.addr = this.addr;
   copy.data = this.data;
   copy.pkt = pkt.copy;
   return copy;
endfunction
endclass
```



深复制就是创建一个新的和原始句柄指向的内容相同的字段，是两个一样大的数据段，所以两者的句柄指向的空间是不同的，但内容是相通的，之后的新对象中句柄指向的内容发生改变，不会引起原始对象中句柄指向内容也发生改变。

对象的销毁

•软件编程的灵活在于可以动态地开辟使用空间，在资源闲置或者不再需要时，可以回收空间，这样使得内存空间保持在一个合理的区间。

•SV也采用了自动回收空间的处理方式，使得用户不再软件空间的开销而烦恼。那么，自动回收空间的基本原理是什么呢？即，当一个对象，在整个程序中没有任何一个地方再‘需要’它时，便会被‘销毁’，即回收其空间。这里‘需要’的意思即指的是有句柄指向该对象。

即特电子Coachip

练习

```
class Transaction;
  logic [31:0] addr,data[8];

  function new(logic[31:0]a=3,d=5);
      addr=a;
      foreach(data[i])
            data[i]=d;
  endfunction
endclass
```

```
Transaction tr,t1;

initial begin
    tr = new(10,1);
    t1 = new(5,2);
    $display ("Addr= %d",t1.addr);
    t1 = tr;
    $display ("Addr= %d",t1.addr);
    t1.addr= 6;
    $display ("Addr= %d",tr.addr);

end
```

叩特电子Coachip

Q&A

昂特电子Coachip